简历用项目经历

> 按「项目名称 + 时间 + 角色 / 技术问题 / 解决方案 / 最终成果 / 技术栈」组织，可直接粘贴进简历。数据均为实测，可复现。

本地优先的 AI 日程助手（端侧 LLM 结构化工具调用 + 规划求解）

2026.09 - 至今 | 独立开发（算法 / 后端 / 客户端 / 工程化 全栈）

技术问题

1. **端侧小模型的结构化输出不可靠。**0.6B–4B 模型自由生成 JSON 时格式错误、字段漂移， 无法直接驱动工具执行（解析失败率是 0 容忍的）。
2. **语言模型做算术必错。**让模型计算”下周三”的绝对日期，跨月、跨年、周偏移场景系统性出错， 而且错误是**静默的**——错一周和错一天在界面上看不出区别。
3. **推理延迟与交互体验冲突。**Qwen3 默认开启 thinking，0.6B 在核显上端到端 1900ms， 4B 甚至把 token 预算全花在 reasoning 上、正文字都不输出。
4. **多消费方导致契约漂移。**同一套数据结构要同时喂给约束解码语法、训练标签、执行层校验、MCP 工具定义；任一处手写就会逐渐不一致，训练与推理悄悄错位。
5. **训练数据无法人工标注。**工具调用样本需要精确的结构化标签，人工标注成本高、错误不可控。
6. **二阶段”帮我安排这周”是规划问题，不是语言问题。**让 LLM 直接产出可行计划， 在公认基准上准确率仅三成量级，且不可解释、不可复现。

解决方案

1. **约束解码（Constrained Decoding）**：把 JSON Schema 编译成 GBNF 语法， 推理时按语法逐 token 约束采样，格式合法率从”需要重试”提升到 **4372/4372 全部合规。** 同时在执行层保留确定性校验与 `clarify / reject` 分支，不让模型”编造可执行意图”。

2. **模型不算日期，只做语义归一化。**契约里只允许相对表达 （`date_kind` / `week_offset` / `month_offset`），绝对时间由确定性代码解析。 彻底消除”模型算术”这一整类不可测错误。
3. **定位并消除思考模式的开销**：实验证明 `--reasoning-budget 0` 无效（只把文本挪到 content 字段、模型仍在乱想），改用 `--reasoning off` 后**输出逐字一致**， 端到端 **1900ms → 641ms（3.6×）**，首字延迟 2067ms → 48ms。
4. **单一事实源（Single Source of Truth）**：一份 `JSON Schema` 派生四处消费 —— GBNF 语法 / LoRA 训练标签 / 执行层 Pydantic 校验 / **MCP 工具 `inputSchema`**。CI 断言”Pydantic 镜像与 Schema 零漂移”，并断言 MCP 工具枚举与 Schema 完全相等。 MCP 侧用 `Literal[*SCHEMA_ENUM]` 直接取枚举，改 Schema 即自动同步。
5. **反向构造数据合成**：先采样结构化真值（几件事、各自时长、截止、排期约束），再渲染成自然中文，**标签由构造保证 100% 正确**，无需人工标注。 另写独立校验器逐条断言”标签每个字段都能从文本推断”，并用自测证明校验器 确实能抓到错误（人为注入五类错标，9/9 全部捕获、干净数据 0 误报）。
6. **三层架构把规划交给求解器**：理解层（LLM 抽取待办+约束）→ 求解层 **OR-Tools CP-SAT** 求可行周计划 → 表达层模板渲染。求解仅 **21–32ms**； 不可行时**软化硬约束重解**，输出”是哪几条约束在冲突”而非”排不出来”。 并设计 diff-only 局部重排保证已确认安排不被无故改动。

最终成果

- **性能**：本机核显（无独显）端到端中位 **641ms**；吞吐 0.6B 111.5 / 1.7B 51.5 / 4B 22 tok/s
- **可靠性**：约束解码 **4372/4372** 样例 schema 合规；契约漂移 **0**；数据集生成端到端失败 **0**
- **测试**：规划器 22/22、规则层 37/37、日期解析 29/29、鉴权 12/12、主题一致性 10/10
- **规划质量**：CP-SAT 求解 21–32ms，不可行时可解释到具体约束
- **抽取能力（二阶段）**：留出集 100 条带置信区间评估； 引入确定性规则层后**整条全对 4% → 39%**、截止时间 52.6% → 100%、排期约束 30.6% → 94.4% （“让规则干规则擅长的事”，与”模型不算日期”同一原则的延伸）

- **交付**：Web + Android（Kotlin / Compose / Material 3）双端；MCP 服务端 4 个工具；开源 MIT，GitHub Release v0.1.0（GGUF 权重 + LoRA adapter + APK，SHA256 校验）
- **工程化**：GitHub Actions 全绿；CI 覆盖 BOM 检查、密钥审计、隐私审计、 契约漂移、数据集标签校验、MCP 契约一致性

技术栈

- **推理**：llama.cpp（Vulkan 后端）、GGUF 量化（Q4_K_M / Q8_0）、Qwen3 0.6B/1.7B/4B
- **结构化输出**：JSON Schema → GBNF 约束解码；Pydantic 契约镜像
- **微调**：LoRA / PEFT、transformers、反向构造数据合成
- **规划求解**：OR-Tools **CP-SAT**（区间变量、NoOverlap、软硬约束、冲突解释）
- **协议**：**MCP（Model Context Protocol）**，工具定义由 Schema 派生
- **后端**：FastAPI、SQLite、Uvicorn
- **客户端**：Android（Kotlin、Jetpack Compose、Material 3、DataStore）、Web（原生 JS）
- **工程**：GitHub Actions、pytest 风格自研断言集、PowerShell/Bash 运维脚本

一句话版本（简历摘要区可用）

独立设计并落地一套**端侧 LLM 工具调用**系统：以”一份 JSON Schema 驱动约束解码 / 训练标签 / 执行校验 / MCP 工具定义”消除契约漂移，以”模型不算日期”消除整类算术错误， 用 CP-SAT 替代 LLM 做规划；在无独显的核显上做到端到端 641ms， 约束合规 4372/4372，并开源双端客户端与 MCP 服务端。

关于 AI 辅助开发的实践（可选一行）

以 Agent 化开发环境（DeepSeek Harness）驱动实现，核心做法是**把每个判断都写成可执行断言** ——测试、CI 门禁、契约一致性检查、数据集标签校验——使 Agent 的每次改动都能被机器验证， 而不是靠人工 review 兜底；项目全程 18 次提交、CI 始终全绿。